that attribute / field and that will decide the kind of database column that will be created to store that data.

We can also specify some 'Meta' options for the class that will change the way the class behaves in some cases. For instance we can configure extra permissions for this model in the meta options. We can also specify a name for the database table for this model if you don't like the one that is automatically generated.

Let's create a model for our Article class to see this in action. The following code for 'models.py' contains a model class that showcases fields, metadata and methods:

```
class Article(models.Model):
    title = models.CharField(max_length=255)
    slug = models.SlugField()
    author = models.ForeignKey(User)
    content = models.TextField()
    creation_date = models.DateTimeField(auto_now_add=True)
    edit_date = models.DateTimeField(auto_now=True)
    featured_image = models.ImageField(null=True, blank=True)
    publish_status = models.BooleanField( default=False)

    def __str__(self):
        return self.title

    class Meta:
        verbose_name_plural = 'Articles'
        ordering = ['creation_date']
```

The above code creates an Article model with:

- A character field for the title that with a maximum length of 255 characters
- A slug field that will simply have a cleaner version of the title
- An author field that uses Django's ForeignKey field to link to a 'User' which is another model and another database table
- A content field that is a TextField for large volumes of text.
- A creation date that is automatically set to the time and date when the article is added
- An edit date that is automatically set to the time and date whenever the article is edited
- A featured image field, that has 'null' and 'blank' set to true, which